

1. **Descrição do Problema e Público-Alvo** O problema escolhido é a dificuldade que pequenos comerciantes enfrentam para realizar o controle de entrada e saída de mercadorias, muitas vezes dependendo de cadernos ou planilhas desorganizadas. O público-alvo engloba pequenos empreendedores, lojistas e autônomos que necessitam de uma solução rápida e intuitiva para registrar seus produtos e evitar a falta de mercadorias.

2. **Requisitos Funcionais** O sistema deve permitir o cadastro de novos produtos com nome, quantidade e preço.

O sistema deve listar todos os produtos cadastrados em uma interface visual.

O sistema deve permitir a alteração de dados de um produto selecionado.

O sistema deve permitir a exclusão de produtos obsoletos.

O sistema deve exportar os dados do estoque para um arquivo JSON para fins de backup.

O sistema deve fornecer um relatório de "Estoque Baixo" para auxiliar a tomada de decisão em compras.

3. **Principais Telas do Sistema** O sistema foi projetado em uma interface de tela única (Single-Window Application) utilizando Tkinter para otimizar o fluxo de trabalho. A tela é dividida em três seções principais:

Formulário de Entrada: Campos para digitação de Nome, Quantidade e Preço. Painel de Ações: Botões para Adicionar, Atualizar, Excluir, Limpar Campos, Exportar JSON e Filtrar Estoque Baixo. Tabela de Visualização (Treeview): Uma grade interativa exibindo os registros em tempo real.

4. **Abordagem RAD no Desenvolvimento** A metodologia RAD (Rapid Application Development) acelerou o desenvolvimento ao permitir a prototipagem rápida da interface gráfica com o Tkinter. Ao invés de planejar exaustivamente toda a arquitetura de antemão, a interface foi construída em paralelo com as funções de banco de dados, permitindo testar a usabilidade e o fluxo de dados (como o tratamento de exceções em inputs vazios ou inválidos) de forma iterativa e incremental.

Para a Persistência Dos Dados

Persistência e Arquivos Para a persistência dos dados, o sistema adota o SQLite, um banco de dados relacional embutido e nativo na biblioteca padrão do Python.

Decisão de Armazenamento: A escolha do SQLite se justifica pelo contexto da aplicação local. Ele não exige a configuração de um servidor dedicado, permitindo que a aplicação seja "plug-and-play". A tabela produtos foi estruturada para suportar as operações CRUD de maneira eficiente.

Importação/Exportação: Foi implementado o recurso de exportação utilizando arquivos JSON. O JSON foi escolhido por ser um formato leve e de fácil leitura para sistemas modernos, facilitando integrações futuras caso o pequeno lojista deseje migrar seus dados para um sistema web no futuro. O recurso utiliza a biblioteca nativa json para transformar os dados estruturados do banco de dados em um formato portátil.

Avaliação Técnica

Análise Crítica do Desenvolvimento O desenvolvimento desta aplicação de Gerenciamento de Estoque proporcionou uma visão prática consolidada da união entre persistência de dados e interfaces gráficas com Python, elementos centrais da metodologia RAD.

Dificuldades Encontradas: O principal desafio durante a construção da interface utilizando o framework Tkinter foi o alinhamento e o gerenciamento do layout (utilizando o grid e pack). Além disso, garantir que os dados selecionados na Treeview fossem extraídos e repassados de forma precisa para os campos de edição exigiu atenção à manipulação de eventos da interface (). **Aprendizados:** A implementação evidenciou a importância crítica do tratamento de exceções na interação com o usuário. Ao construir a função validar_entradas(), ficou claro que usuários podem digitar textos em campos numéricos ou deixar campos em branco. O tratamento via blocos try...except associado ao uso de Messageboxes melhorou substancialmente a robustez do software, impedindo que erros não tratados fizessem o programa fechar abruptamente.

Relação entre Teoria e Prática: O desenvolvimento incremental, um pilar da metodologia RAD, mostrou-se altamente prático. Foi possível testar módulos isolados (como a classe do Banco de Dados) antes de conectar toda a lógica com os botões visuais. O isolamento da lógica do banco de dados (BancoDeDados) e da lógica de interface (AppEstoque) utilizou conceitos primários de Orientação a Objetos, garantindo um código mais limpo.

Melhorias Futuras: Em iterações futuras, a aplicação poderia ser expandida para contemplar a importação de lotes de produtos via arquivos CSV/JSON. Além disso, a adição de recursos visuais de relatórios, como a geração de gráficos de vendas ou estatísticas de movimentação utilizando bibliotecas analíticas como Matplotlib ou Pandas, agregaria ainda mais valor ao processo de tomada de decisão do usuário final.

"Seguindo estritamente as premissas da metodologia RAD (Desenvolvimento Rápido de Aplicações), após a consolidação das regras de negócio e validação dos requisitos funcionais do CRUD, o sistema passou por um ciclo de refatoração incremental focado em UI/UX. A interface cinza nativa do framework Tkinter foi substituída por uma identidade visual contemporânea no estilo 'Flat Design', utilizando uma paleta de cores corporativa (Slate e Midnight Blue), tratamento dinâmico e formatação monetária (padrão R\$) na visualização dos dados da Treeview, e uma barra de status responsiva

para feedback em tempo real das interações do usuário. Essa evolução demonstra como o ciclo RAD permite polir e profissionalizar um protótipo funcional em um curto espaço de tempo."

Algumas imagens do projeto funcionando abaixo:

